

ShadowHeal

Self-Diagnosing & Self-Healing Observability Platform

Complete Engineering Blueprint · Architecture, Layers & Phased Build

Pip SDK

one-line install

5 Agents

LangGraph graph

3 Signals

errors · N+1 · perf

Human-gated

fixes need approval

ShadowHeal is a distributed observability platform that installs into any Python backend as a single pip-installable SDK. It watches the running application, catches errors, exceptions, N+1 queries, and performance regressions, and surfaces an instant warning in the developer's terminal. When the developer asks, a multi-agent AI engine diagnoses the true root cause using the project's own code and history as context, explains it in plain language, and — only on explicit approval — writes a candidate fix, tests it, and places it back into the editor for the developer to review before anything ships.

Prashanth Hosamani

B.Tech CSE · Lovely Professional University · 2027

Contents

01	The problem ShadowHeal solves
02	Product philosophy — passive until it matters
03	The full system — 6 layers end to end
04	Master architecture diagram
05	Layer 1 — ShadowHeal Agent SDK
06	Layer 2 — Ingestion & Gateway API
07	Layer 3 — Queue & async processing
08	Layer 4 — The AI diagnosis engine (5 agents)
09	Layer 5 — GitHub read-only context
10	Layer 6 — Persistence, dashboard & platform
11	The fix workflow — how healing stays safe
12	Data model & schema
13	Phase 1 — Capture & warn
14	Phase 2 — Diagnose & explain
15	Phase 3 — GitHub context & RAG
16	Phase 4 — Guarded auto-fix
17	Phase 5 — Dashboard & platform
18	Phase 6 — Production on AWS
19	Cross-cutting: cost, tracing, security, tests
20	Edge cases — the full catalogue
21	How to talk about it in interviews
22	Resume bullets

01 The problem ShadowHeal solves

Every backend developer loses hours to the same loop. Something breaks in a running application — a 500 error, a request that suddenly takes three seconds, an endpoint firing two hundred database queries. The developer sees a symptom, not a cause. They dig through logs, try to reproduce it locally, guess at the reason, and slowly narrow it down. This cycle repeats dozens of times a week and it is almost entirely undifferentiated grunt work.

Existing tools help but stop short. Sentry tells you an error happened and gives you a stack trace. Datadog shows you a latency graph. Neither tells you *why* it happened in the context of your specific codebase, and none of them propose an actual fix. The developer is still the one who has to reason from symptom to cause to solution.

ShadowHeal closes that loop. It captures the same signals Sentry does, but then goes three steps further: it reasons about the root cause using the project's own source code as context, explains that cause in plain language, and — only when the developer explicitly asks — writes a candidate fix, tests it, and hands it back for human review. It never changes code on its own. The developer stays in control at every irreversible step.

The one-sentence pitch

- Sentry tells you something broke. ShadowHeal tells you why, in the context of your code, and — on your command and never before — proposes a tested fix you review before it ships.

02 Product philosophy — passive until it matters

The entire design rests on one principle: **silent during normal work, active only when something breaks, and never destructive without explicit human approval**. A developer integrates the SDK once and then forgets it exists. It produces no noise during a normal day. The moment a real problem occurs, it surfaces exactly one concise warning and waits.

Three hard rules follow from that principle, and they shape every technical decision in this document:

- **The SDK never slows the app.** Capture and transmission happen on a background thread. If ShadowHeal's own backend is down, the developer's application does not notice.
- **Diagnosis is pull, not push.** The system detects and warns automatically, but the deep AI diagnosis runs only when the developer asks for it — via a terminal command or by opening the platform. This controls cost and respects attention.
- **Healing is gated.** The system will draft a fix, test it, and place it in the editor. It will never commit, push, or overwrite anything on its own. A human reviews and applies every change.

What ShadowHeal is NOT

- Not an autonomous agent that rewrites your production code unsupervised — every fix is human-approved.
- Not a monitoring dashboard you have to constantly watch — it comes to you, once, when it matters.
- Not a wrapper around ChatGPT — the value is the orchestration, the code-aware context, and the safety gates around the model.

03 The full system — 6 layers end to end

ShadowHeal is six layers, each with a single responsibility. Data flows down through them when a problem occurs and results flow back up to the developer. Every file you build lives in exactly one of these layers — when you feel lost mid-build, come back here and ask which layer you are in.

- 1

Layer 1 — Agent SDK. Pip-installed into the developer's app. Hooks the framework, captures errors / N+1 / slow requests, prints a terminal warning, ships events to the backend on a background thread.
- 2

Layer 2 — Ingestion API. FastAPI. Authenticates the project by API key, validates the event, rate-limits, and pushes it onto the queue. Returns instantly.
- 3

Layer 3 — Queue & workers. Redis holds events, Celery workers consume them. All heavy analysis happens here, off the request path.
- 4

Layer 4 — AI diagnosis engine. A LangGraph supervisor routes the event through five agents: Triage, Context, Diagnosis, Fix, and Evaluation.
- 5

Layer 5 — GitHub context. A read-only GitHub integration lets the Context agent pull the exact source files involved, grounding diagnosis in the real code (RAG over the codebase).
- 6

Layer 6 — Persistence & platform. PostgreSQL stores issues, diagnoses, traces, and fix proposals. A web dashboard shows every incident, explanation, and proposed fix across all of the developer's projects.

The developer experiences only two touch points: a one-line warning in their terminal the instant something breaks, and — when they choose — a rich explanation and optional fix, either inline via a command or on the web platform. Everything between those two touch points is the engineering this document describes.

04 Master architecture diagram

The full path of a single incident, from the moment code breaks to the moment a fix is proposed:

```
DEVELOPER'S RUNNING APP
( FastAPI / Django + ShadowHeal SDK installed )
|
| exception / N+1 / slow request detected
v
[L1] SDK --- prints terminal warning instantly
| --- ships event on background thread (non-blocking)
v
[L2] INGESTION API (FastAPI)
| auth by API key -> validate -> rate-limit -> enqueue
v
[L3] REDIS QUEUE -> CELERY WORKER
|
v
[L4] LANGGRAPH SUPERVISOR
|-- Triage agent (classify, dedupe, severity)
|-- Context agent ---> [L5] GITHUB read-only (pull real code)
|-- Diagnosis agent (root cause + plain explanation)
|-- Fix agent (ONLY if developer approved)
|-- Evaluation agent (score the diagnosis / fix)
v
[L6] POSTGRES (issues, diagnoses, traces, fix proposals)
|
+--> TERMINAL (developer types a command to see cause / request fix)
+--> WEB DASHBOARD (all incidents, explanations, proposed fixes)
```

Read that flow three times before writing any code. The two developer touch points sit at the very bottom — the terminal and the dashboard. Everything above them is invisible to the user and is where all the engineering lives.

05 Layer 1 — ShadowHeal Agent SDK

Plain English. A small Python package the developer installs once. It quietly hooks into their framework, and whenever an error, an N+1 query pattern, or a slow request happens, it does two things: prints a short warning in the terminal immediately, and sends the details to the ShadowHeal backend on a background thread so the app is never slowed down.

Install & integrate

```
pip install shadowheal

# FastAPI
from shadowheal import ShadowHeal
ShadowHeal.init(
    api_key="proj_live_xxx",
    project="checkout-service",
    environment="production",
)

# Django -> add to settings.py, same init() call
```

What it captures

Signal	How it is detected	Example
Exception	Framework error hook / middleware	OperationalError: column missing
Slow request	Request duration > threshold (e.g. 2000ms)	GET /products took 3412ms
N+1 query	Query count per request > threshold (e.g. 50)	213 queries on one request
Auth failure	Repeated 401 / malformed token pattern	Invalid Bearer token

The hybrid terminal experience

When something breaks, the developer sees a concise, non-intrusive warning. Crucially, the deep diagnosis does not run automatically — the warning tells them the command to run if they want it, and that same incident is already waiting on the platform.

```
[ShadowHeal] ■ N+1 query detected on GET /api/products (213 queries)
Issue id: iss_9fa22c
Explain : run `shadowheal explain iss_9fa22c`
Or view : https://app.shadowheal.io/issues/iss_9fa22c
```

The explain command is what triggers the AI engine. Detection is automatic and cheap; diagnosis is on-demand and is where the LLM cost is spent. This is the 'pull, not push' rule from the philosophy section, expressed concretely.

SDK edge cases

- SDK's own code throws — every path wrapped in broad try/except; SDK failure never reaches the host app.
- Backend unreachable — retry 3x with backoff on the background thread, then drop silently. App never blocks.
- Same error fires 500 times in a loop — local fingerprint cache suppresses duplicate sends within a 60s window.
- init() never called — every hook becomes a no-op; the app runs exactly as if ShadowHeal were not installed.

06 Layer 2 — Ingestion & Gateway API

Plain English. A FastAPI service that receives events from every SDK in the wild. Its only job is to accept an event fast: confirm the project is real, check the data is well-formed, make sure one noisy project cannot flood the system, drop the event on the queue, and reply immediately. It never analyses anything itself.

Why FastAPI and not Django here

- Ingestion is pure I/O — receive, validate, enqueue. Async handles far more concurrent connections at the same cost.
- Pydantic validates every payload automatically and rejects malformed events before your code runs.
- No ORM overhead on the hot path — the ingestion endpoint does not touch the main database at all, only Redis.

Responsibilities

Responsibility	How
API-key auth	X-API-Key header checked against the projects table; unknown key -> 401
Rate limiting	Redis sliding-window counter per project; over limit -> 429
Schema validation	Pydantic model with extra='forbid'; bad shape -> 422
Enqueue	Valid event pushed to the Redis list shadowheal:events
Instant reply	202 Accepted returned in < 50ms; analysis is asynchronous

```
@router.post('/v1/events/ingest', status_code=202)
async def ingest(payload: EventIn,
api_key: str = Header(alias='X-API-Key'),
redis: Redis = Depends(get_redis),
db: AsyncSession = Depends(get_db)):
    await validate_api_key(api_key, db)
    await check_rate_limit(payload.project_id, redis)
    await redis.lpush('shadowheal:events', payload.json())
    return {'status': 'accepted'}
```

Ingestion edge cases

- Burst of 5000 events/sec — rate limiter caps per project; excess gets 429 and the SDK backs off.
- Malformed / unknown fields — Pydantic extra='forbid' rejects with 422, protecting against SDK version drift.
- Redis down — return 503; SDK retries. Some events may be lost during an outage; acceptable for a dev tool.
- API key from a deleted project — lookup returns nothing -> 401, stopping orphaned SDKs from flooding in.

07 Layer 3 — Queue & async processing

Plain English. Events must never be analysed while the ingestion endpoint waits — diagnosis can take seconds, ingestion must answer in milliseconds. Redis holds the events; Celery workers pick them up and do the slow work in the background. Add more workers to handle more load — no code change.

Why async is non-negotiable

- Ingestion must answer in < 50ms; AI diagnosis can take 2–30s. The queue decouples the two.
- Workers scale horizontally — more load just means more worker containers.
- A failed analysis can be retried without losing the event.
- Redis doubles as the deduplication cache (fingerprint -> last-seen) and the rate-limit store.

The worker pipeline

- 1 Pop an event from the Redis queue (blocking pop — zero CPU while idle).
- 2 Deduplicate — has this exact fingerprint been analysed in the last few minutes? If so, just bump its count.
- 3 Fingerprint — normalise the traceback into a stable hash so repeat occurrences group into one issue.
- 4 Persist the raw issue to PostgreSQL as 'detected' (this is what powers the terminal warning + dashboard).
- 5 Stop here in Phase 1. In later phases: if diagnosis was requested, invoke the LangGraph engine.

The critical detail: **detection and diagnosis are separate steps**. Every event is detected, fingerprinted, and stored cheaply. Only events the developer explicitly asks about get the expensive LLM diagnosis. This split is what keeps the whole system affordable.

08 Layer 4 — The AI diagnosis engine (5 agents)

Plain English. This is the brain, and it only runs when a developer asks for a diagnosis. A supervisor reads the issue and routes it through five specialists in turn, each doing one job and writing its result into a shared state object. This is a LangGraph state machine — the same pattern real production agent systems use.

The five agents

Agent	One job	Key output
Triage	Classify the issue, judge severity, decide if diagnosis is even needed	issue_type, severity
Context	Pull the exact source files involved from GitHub (read-only) via RAG	relevant_code, file paths
Diagnosis	Reason about root cause using error + code; explain in plain language	root_cause, explanation
Fix	ONLY if approved: draft a patch, run it against tests in a sandbox	patch, test_result
Evaluation	Score the diagnosis/fix for confidence; flag low-confidence output	scores, flagged

The supervisor & state

```
class HealState(TypedDict):
    issue_id: str
    error_type: str
    traceback: str
    relevant_code: Optional[str]
    root_cause: Optional[str]
    explanation: Optional[str]
    fix_requested: bool # gate – set only by explicit command
    patch: Optional[str]
    scores: Optional[dict]
    next_agent: str
    step_count: int # loop guard – never remove
```

Routing is mostly deterministic Python, not an LLM call: Triage first, then Context, then Diagnosis, then stop — unless `fix_requested` is true, in which case the Fix agent runs before Evaluation. Using an LLM only for the genuinely fuzzy judgement (classifying the error) and plain `if` statements for everything else makes the system cheaper, faster, and far easier to debug.

The Context agent — why this is the whole differentiator

A plain LLM guesses at causes from a stack trace alone. The Context agent instead pulls the *actual source file and line* named in the traceback from the developer's own GitHub repo (read-only), embeds surrounding code, and hands the Diagnosis agent the real code — not a guess about what the code might look like. Grounding the diagnosis in the true source is exactly what separates ShadowHeal from someone pasting an error into ChatGPT.

The Evaluation agent — catching confident wrong answers

After a diagnosis (or a fix) is produced, a second model scores it: does the explanation actually follow from the code and error (faithfulness), and does the proposed fix plausibly address the named cause (relevance)? Anything below threshold is flagged to the developer as low-confidence rather than presented as certain. Most portfolio projects can produce an answer; almost none can tell you whether the answer is trustworthy. This agent is the piece interviewers remember.

Agent-engine edge cases

- Supervisor loops — `step_count` cap forces termination and returns whatever was produced so far.
- An agent throws — caught inside the node, error written to state, control returns to supervisor to decide.
- GitHub file not found (code changed since deploy) — Context agent degrades gracefully, diagnoses from the trace alone and lowers its confidence.
- Diagnosis low-confidence — Evaluation flags it; the developer sees 'best guess' framing, never false certainty.

09 Layer 5 — GitHub read-only context

Plain English. To diagnose an error properly, the system needs to see the real code that broke. The developer connects their GitHub with **read-only** permission. When an error names a file and line, the Context agent fetches that file (and nearby related files) so the diagnosis is grounded in reality instead of guesswork.

Why read-only, and why it matters

- **Read-only is a hard boundary.** ShadowHeal can read source to understand it; it can never push, commit, or alter the repository. The OAuth scope itself forbids writes — this is enforced by GitHub, not by a promise in a prompt.
- This is a RAG problem over code. Files are chunked, embedded, and stored so the Context agent can retrieve the most relevant snippets for a given error, not just the single named file.
- It also gives cross-project knowledge: because ShadowHeal can see the developer's repos, it can recognise patterns that repeat across their projects.

The flow

- 1 Developer connects GitHub once, granting read-only repo access via OAuth.
- 2 On first connect, ShadowHeal indexes the repo: chunk source files, embed them, store vectors in the vector DB.
- 3 When an error arrives, the Context agent extracts the file+line from the traceback.
- 4 It retrieves that file plus the most semantically related chunks (RAG) for surrounding context.
- 5 It hands that real code to the Diagnosis agent, which now reasons about actual source, not a guess.

GitHub-integration edge cases

- Private repo, token expired — catch the auth error, fall back to trace-only diagnosis, prompt the user to reconnect.
- The error's file has changed since deploy — line numbers may not match; retrieve by symbol/function name, not just line.
- Huge monorepo — index lazily per-project and cap embedding volume; never block on indexing the whole org.
- Write attempted anywhere in code — impossible by construction; the granted scope has no write permission at all.

10 Layer 6 — Persistence, dashboard & platform

Plain English. Everything ShadowHeal learns is stored so the developer can see it. PostgreSQL holds every issue, diagnosis, agent trace, and fix proposal. The web dashboard is where a developer sees all incidents across all their projects, reads full explanations, and reviews proposed fixes — the richer counterpart to the one-line terminal warning.

What the dashboard shows

- A live list of all incidents across every connected project, grouped by fingerprint with occurrence counts.
- For each issue: the full plain-language explanation, the exact code the diagnosis was based on, and the confidence score.
- For issues where a fix was requested: the proposed patch, the sandbox test result, and an explicit 'this was reviewed by a human' state — never auto-applied.
- Trends per project — is this error recurring, is latency creeping up over time.

This is also where a developer who does not know the terminal command can simply open the project, see the detected error already waiting, and click to get the explanation — the alternative entry point into diagnosis mentioned in the philosophy.

11 The fix workflow — how healing stays safe

This is the most sensitive part of the whole system and its safety is a feature, not a limitation. ShadowHeal will help fix an error, but the human is in control at every irreversible step. Here is the exact sequence.

- 1

Error is detected and stored. The developer sees the warning. **Nothing else happens automatically.**
- 2

Developer requests an explanation (command or dashboard). The AI engine diagnoses and explains the root cause.
- 3

Only if the developer then explicitly requests a fix does the Fix agent run. Silence means no action.
- 4

The Fix agent drafts a candidate patch and runs it against the project's tests in an isolated sandbox.
- 5

The patch + test result are placed back into the developer's editor (e.g. VS Code) as a suggested change.
- 6

ShadowHeal explicitly tells the developer to review and test it themselves before applying — it does not apply it.
- 7

The developer accepts, edits, or rejects. ShadowHeal never commits, pushes, or overwrites on its own.

The safety guarantees, stated plainly

- No fix is ever generated without an explicit developer command — approval is opt-in, never opt-out.
- A sandbox can prove a patch compiles and passes tests; ShadowHeal never claims it 'verified the production fix'.
- The patch lands in the editor as a suggestion, not in the repo as a commit. The human applies it.
- This gate is the honest answer to 'isn't auto-fixing dangerous?' — yes, which is why it is never autonomous.

An honest note on limits, worth stating in an interview: generating a *correct* patch for a real production bug is genuinely hard, and a sandbox cannot fully reproduce live traffic and real database state. ShadowHeal therefore frames every fix as a reviewed *candidate*, and the whole workflow is designed around the assumption that the model may be wrong — which is why a human gates the apply step.

12 Data model & schema

Every table is scoped to a project, and every id is a UUID so nothing can be enumerated. JSONB is used for nested, evolving payloads like tracebacks and agent traces.

projects

id (UUID PK) · owner_id (FK users) · name · api_key (unique, indexed) · github_repo · environment · created_at

issues

id (UUID PK) · project_id (FK, indexed) · fingerprint (indexed) · issue_type (indexed) · traceback (TEXT) · file_path · line_no · occurrence_count · first_seen · last_seen · status (detected/diagnosed/fix_proposed/resolved)

diagnoses

id (UUID PK) · issue_id (FK) · root_cause (TEXT) · explanation (TEXT) · relevant_code (TEXT) · confidence (FLOAT) · engine (rule/llm) · created_at

fix_proposals

id (UUID PK) · issue_id (FK) · patch (TEXT) · test_result (JSONB) · sandbox_passed (BOOL) · applied_by_human (BOOL default false) · created_at

agent_traces

id (UUID PK) · issue_id (FK, indexed) · agent_name · input (JSONB) · output (JSONB) · tokens · duration_ms · error · created_at

evaluations

id (UUID PK) · issue_id (FK) · faithfulness · relevance · flagged (indexed) · reasoning · created_at

Schema decisions worth explaining

- UUIDs not integers — sequential ids let anyone enumerate other projects' issues; UUIDs remove that class of bug.
- fingerprint indexed on issues — every new event upserts by fingerprint; without the index that is a full scan per event.
- traceback is TEXT not VARCHAR — Python tracebacks routinely exceed any fixed length; VARCHAR would truncate silently.
- applied_by_human on fix_proposals — the schema itself encodes that a fix is only ever applied by a person.

The phased build

ShadowHeal is built in six phases. Each phase produces something that actually runs and is worth showing on its own. Build a phase, run it, understand every line, then move on. Never stack a half-built phase on another half-built phase — that is how projects die. Each phase below has its own focused architecture so you always know exactly what you are assembling.

The single most important rule: **write the plain-English description of each component first, attempt the code yourself, then check it.** The moment you paste in code you did not reason through, the knowledge stops sticking.

PHASE 1

Capture & warn

the foundation

Goal. An SDK that catches an error in a real app and a backend that receives, stores, and can show it. No AI yet. This alone proves the hardest plumbing works.

Phase 1 architecture

```
APP + SDK -> FastAPI ingest -> Redis -> Celery worker -> Postgres
|
`shadowheal list` CLI
```

Build order

- | | |
|---|---|
| 1 | Postgres schema: projects + issues tables. Migrate. Seed one test project with an API key. |
| 2 | FastAPI ingest endpoint: Pydantic validation + API-key auth + push to Redis. Return 202. |
| 3 | Celery worker: pop event, fingerprint it, upsert into issues. Nothing more. |
| 4 | SDK v0.1: pip-installable, FastAPI exception hook, prints terminal warning, sends event on a background thread. |
| 5 | A tiny CLI: `shadowheal list` shows recent issues for the project. Docker Compose ties it all together. |

Phase 1 done when

- You trigger a real error in a sample FastAPI app, see the terminal warning instantly, and see the issue appear in Postgres via the CLI — end to end, with `docker compose up`.

PHASE 2 Diagnose & explain

add the brain

Goal. On demand, run a diagnosis that explains an error's root cause in plain language. Introduces LangGraph and the first agents — but still diagnoses from the traceback alone (code context comes in Phase 3).

Phase 2 architecture

```
`shadowheal explain ` -> API -> Celery -> LangGraph
|-- Triage agent
|-- Diagnosis agent (trace-only)
`-- Evaluation agent
|
Postgres (diagnoses table)
```

- 1 Add a diagnoses table + the diagnosis request flow (a command sets a 'diagnose me' flag on the issue).
- 2 Build the smallest LangGraph: supervisor + one Diagnosis node. Get state flowing and printing.
- 3 Diagnosis agent: feed it the traceback, get a structured root_cause + plain explanation (Pydantic output).
- 4 Add the Triage agent (classify + severity) and the Evaluation agent (score faithfulness, flag low confidence).
- 5 Wire `shadowheal explain ` to trigger it and print the explanation back in the terminal.

Phase 2 done when

- `shadowheal explain ` returns a real, plain-language root-cause explanation with a confidence score — and low-confidence answers are visibly flagged rather than presented as certain.

PHASE 3

GitHub context & RAG

ground it in real code

Goal. Make diagnosis dramatically better by giving it the real source code. This is the RAG-over-code phase and the Context agent.

Phase 3 architecture

```
GitHub (read-only OAuth) -> index repo -> Vector DB (code chunks)
^
LangGraph: Triage -> Context agent --RAG--> Diagnosis -> Evaluation
```

- 1 Read-only GitHub OAuth connect flow; store the repo reference on the project.
- 2 On connect, index the repo: chunk source files, embed, store in the vector DB.
- 3 Build the Context agent: from the traceback's file+line, retrieve that file plus related chunks via RAG.
- 4 Feed retrieved real code into the Diagnosis agent — measure how much the explanation improves.
- 5 Handle the degradation path: repo unavailable or file changed -> fall back to trace-only, lower confidence.

Phase 3 done when

- The same error produces a visibly better, code-specific diagnosis with GitHub connected than without it — and you can show the exact retrieved code the diagnosis was based on.

PHASE 4

Guarded auto-fix

the hard, gated part

Goal. On explicit request only, draft a candidate fix, test it in a sandbox, and place it in the editor for human review. This is the most sensitive phase — its whole design is about safety.

Phase 4 architecture

```
`shadowheal fix ` -> Fix agent
|-- draft patch (from diagnosis + real code)
|-- run against tests in ISOLATED sandbox
|-- Evaluation agent scores the patch
`-- place patch in VS Code -> HUMAN reviews & applies
(never auto-commit, never auto-push)
```

- 1
- fix_proposals table + the explicit `shadowheal fix ` command that sets the fix-requested gate.
- 2
- Fix agent drafts a patch from the diagnosis and the real retrieved code.
- 3
- Run the patch against the project's tests inside an isolated sandbox container; capture the result.
- 4
- Evaluation agent scores the patch; store patch + test result + confidence in fix_proposals.
- 5
- Deliver the patch into the editor as a suggested change, with an explicit 'review and test this yourself' prompt.

Phase 4 safety, restated

- The Fix agent runs only after an explicit command — never as part of normal diagnosis.
- The sandbox proves tests pass, not that production is fixed — the framing stays honest.
- The patch is a suggestion in the editor; applied_by_human stays false until a person acts.

PHASE 5

Dashboard & platform

the web experience

Goal. A web platform where a developer sees every incident across every project, reads full explanations, and reviews proposed fixes — the alternative entry point for anyone not using the terminal.

Phase 5 architecture

```
Web dashboard (React/Next) -> ShadowHeal REST API -> Postgres
- all incidents across projects
- full explanation + the code it was based on
- proposed fixes (review-only) + trends
```

1	Auth + project management REST endpoints (JWT). List issues, get issue detail, get diagnosis, get fix proposal.
2	Dashboard: incident list grouped by fingerprint with occurrence counts and severity.
3	Issue detail page: explanation, the exact code used, confidence, and — if present — the proposed fix (review-only).
4	Per-project trend views: recurring errors, latency creep over time.
5	The 'open the platform and click to diagnose' entry point for developers who never touched the CLI.

PHASE 6

Production on AWS

ship it

Goal. Run the whole platform live, securely, on AWS. Start with a single EC2 instance running Docker Compose — honest, understandable, and completely defensible in an interview.

Target architecture

```
Route 53 -> ALB (HTTPS via ACM) -> EC2 (Docker Compose)
|-- api (uvicorn behind nginx)
|-- worker (celery)
`-- dashboard

RDS Postgres (private) · ElastiCache Redis (private) · EBS (vector DB) · Secrets Manager
```

- 1
- RDS Postgres and ElastiCache Redis in private subnets; security groups allow only the EC2 instance.
- 2
- EC2 (t3.medium — agents need memory); Docker + Compose; EBS volume mounted for vector-DB persistence.
- 3
- Secrets in AWS Secrets Manager / SSM — API keys and JWT secret never touch the repo or the AMI.
- 4
- Nginx reverse proxy + HTTPS terminated at the ALB via an ACM certificate.
- 5
- Run alembic migrations as a one-off container on deploy; ship logs to CloudWatch.

Deployment edge cases that actually bite

- OAuth / callback URLs still pointing at localhost — update every one to the production domain (the classic deploy bug).
- DEBUG left true — force it false via env; never rely on a default.
- Vector DB on container-local disk — it dies on reboot; it must live on the mounted EBS volume.
- DB connections exhausted — set pool_size / max_overflow; the default will exhaust RDS under load.

19 Cross-cutting: cost, tracing, security, tests

Cost control

- **Detection is cheap, diagnosis is on-demand.** The single biggest cost lever — the LLM only runs when a developer asks.
- **Model tiering.** Cheap fast model for triage/routing; strong model only for diagnosis and evaluation.
- **Cache by fingerprint.** The same error diagnosed once is cached; repeat occurrences reuse the diagnosis.

Tracing & observability

Every agent writes a row to `agent_traces` before returning — what it received, produced, how long it took, tokens used. Wrap it in a decorator so it is written once. A failing agent writes an error trace and returns control to the supervisor; it never crashes the graph. This table is also the single most impressive thing to show an interviewer.

Security

- GitHub scope is read-only — writes are impossible by construction, not by prompt instruction.
- Fixes are human-gated — nothing is committed, pushed, or applied autonomously.
- Per-project API keys, UUID ids, project-scoped queries — no cross-tenant access is possible.

Testing

- Validation: bad payloads rejected, good ones accepted.
- The fingerprinting logic: same error -> same fingerprint; different errors -> different.
- Supervisor routing and the step-count loop cap (construct a state at the cap, assert it terminates).
- The GitHub read-only boundary and the fix human-gate — security tests that read very well in interviews.

20 Edge cases — the full catalogue

Every serious edge case across the whole system, in one place. Being able to talk through these is what makes the project read as real engineering rather than a demo.

Where	Situation	What happens
SDK	SDK code itself throws	Wrapped in try/except; never reaches host app
SDK	Backend unreachable	Retry 3x w/ backoff on bg thread, then drop; app unaffected
SDK	Same error floods 500x	Local fingerprint cache suppresses dupes for 60s
Ingest	5000 events/sec burst	Per-project rate limit; excess 429, SDK backs off
Ingest	Malformed payload	Pydantic extra='forbid' -> 422 before your code runs
Queue	Worker crashes mid-run	Checkpointter resumes from last completed node
Agents	Supervisor loops forever	step_count cap forces termination
Agents	An agent raises	Caught in node, error to state, back to supervisor
GitHub	Token expired	Fall back to trace-only diagnosis, prompt reconnect
GitHub	File changed since deploy	Retrieve by symbol not line; lower confidence
Fix	Patch fails sandbox tests	Reported as failed; never delivered as 'working'
Fix	Model unsure of fix	Evaluation flags low confidence; framed as a guess
Deploy	Callback URL = localhost	Must be updated to prod domain (classic bug)
Deploy	Vector DB on local disk	Dies on reboot; must live on mounted EBS

21 How to talk about it in interviews

What it is (30 seconds): "A pip-installable observability SDK like Sentry, but it goes three steps further — it diagnoses the root cause using your actual source code as context, explains it in plain language, and on explicit request drafts a tested fix you review before applying. It never changes code on its own."

Why not just use ChatGPT: "A chat with GPT is a person operating a tool — you paste in the error and the code yourself, and you judge whether the answer is right. ShadowHeal is a system: it decides what context to pull, pulls the real code safely, verifies its own output with an evaluation layer, and keeps a full trace of how it got there. GPT is the sentence-generation engine inside one node — everything around it is the engineering."

Why LangGraph not a loop: "I needed conditional routing and persistent state across steps, plus checkpoint recovery if a worker dies. A loop calls agents in a fixed order; the graph lets the supervisor decide the path based on the issue, and resume from the last completed node after a failure."

How do you know the fix is right: "I don't fully — which is exactly why fixes are human-gated and there's an evaluation agent scoring confidence. The sandbox proves tests pass, not that production is fixed. So every fix is a reviewed candidate placed in the editor, never an autonomous change."

How would you scale it: "The bottleneck is the worker, not the API — the API just enqueues. So scale Celery workers horizontally first, no code change. Then LLM rate limits, handled with tiering and fingerprint caching. Postgres read replicas come much later."

22 Resume bullets

ShadowHeal — Self-Diagnosing Observability Platform | [GitHub](#) | [Live](#)

- Built a distributed observability platform with a pip-installable Python **SDK** that instruments Django/FastAPI apps, capturing errors, N+1 queries, and performance regressions, and shipping events to a **FastAPI** ingestion backend on a non-blocking background thread.
- Designed an on-demand AI diagnosis engine using a **LangGraph** multi-agent supervisor (triage, context, diagnosis, fix, evaluation) with **RAG over the user's GitHub source** to ground root-cause analysis in real code, plus an evaluation agent that flags low-confidence output.
- Implemented a **human-gated auto-fix** workflow that drafts and sandbox-tests candidate patches and delivers them to the editor for review — never committing autonomously — backed by async **Celery/Redis** processing, multi-tenant **PostgreSQL**, and **Docker** on **AWS**.

Python, FastAPI, LangGraph, LangChain, ChromaDB, PostgreSQL, Redis, Celery, GitHub API, Gemini API, Pydantic, Docker, AWS

Build one phase at a time. Understand every line before moving on. Write the plain-English description first, attempt the code yourself, then check it — that is what turns this from a project you built into a project you can explain, which is the thing that actually gets you hired.